COMP3018 – Human-Robot Interaction

Week 10 – Facial Expression Detection (Part - 2)

Semester 2, 2025-2026

The purpose of this workshop is to work on a dataset of facial expressions and train a neural network model to recognize emotions. This exercise builds on the theoretical knowledge you learned in the module **COMP3018**.

Exercise 1

Emotion detection is a challenging task, because emotions are subjective. There is no common consensus on how to measure or categorize them. In this exercise, we will learn - in a live demo - how to recognize facial expressions using a trained deep-learning model.

- In the last week, we created and trained a deep-learning model for detecting facial expressions. After running the model for **15 iterations**, we achieved an **accuracy of 68 %** (more iterations can make the accuracy higher). With this accuracy and model, we will try today to make a **live demo** using our PC camera to recognize our facial expressions.
- Download the model parameter files from the DLE, and in the same folder create a new python script called Face_demo.py, which will read the parameter files **model_weights.h5** and **model.json** of the model that we trained in the last workshop.
- In the python file, create a class called FacialExpressionModel that aim at reading the parameters of the trained model.

```python
import numpy as np
import cv2
import tensorflow as tf
from IPython.display import SVG, Image
from livelossplot.inputs.tf_keras import PlotLossesCallback
from tensorflow.keras.models import model_from_json

class FacialExpressionModel(object):
    EMOTIONS_LIST = ["Angry", "Disgust",
                     "Fear", "Happy",
                     "Neutral", "Sad",
                     "Surprise"]
    def __init__(self, model_json_file, model_weights_file):
        # load model from JSON file
        with open(model_json_file, "r") as json_file:
            loaded_model_json = json_file.read()
            self.loaded_model = model_from_json(loaded_model_json)
        # load weights into the new model
        self.loaded_model.load_weights(model_weights_file)
        self.loaded_model.make_predict_function()
    def predict_emotion(self, img):
        self.preds = self.loaded_model.predict(img)
        return FacialExpressionModel.EMOTIONS_LIST[np.argmax(self.preds)]

model = FacialExpressionModel("model.json", "model_weights.h5")
```

- Afterward, create a classe called VideoCamera that aim at using the trained model to predict, live, the emotion of a facial expression using the camera of the PC. This class detects the human face with a rectangle using OpenCV features.

```
facec = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
font = cv2.FONT_HERSHEY_SIMPLEX

class VideoCamera(object):
    def __init__(self):
        self.video = cv2.VideoCapture(0)
    def __del__(self):
        self.video.release()
    # returns camera frames along with bounding boxes and predictions
    def get_frame(self):
        _, fr = self.video.read()
        gray_fr = cv2.cvtColor(fr, cv2.COLOR_BGR2GRAY)
        faces = facec.detectMultiScale(gray_fr, 1.3, 5)
        for (x, y, w, h) in faces:
            fc = gray_fr[y:y+h, x:x+w]
            roi = cv2.resize(fc, (48, 48))
            pred = model.predict_emotion(roi[np.newaxis, :, :, np.newaxis])
            cv2.putText(fr, pred, (x, y), font, 1, (255, 255, 0), 2)
            cv2.rectangle(fr,(x,y),(x+w,y+h),(255,0,0),2)
        return fr

def gen(camera):
    while True:
        frame = camera.get_frame()
        cv2.imshow('Facial Expression Recognization',frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cv2.destroyAllWindows()

gen(VideoCamera())
```

- Run the code, and evaluate if the system will be able to recognize your different facial expressions or not.